

Experiment – 1

Student Name: ADITYA SHARMA

UID: 24BAI70379

Branch: CSE-AIML

Section/Group: 24AIT_KRG_1-A

th

Semester: 5

Date of Performance: 10/7/26

Subject Name: ADBMS

Subject Code: 24CSP-310

(A)

Write an SQL solution to calculate the total number of bank accounts belonging to each salary category. The

classification categories are:

1. "Low Salary": All monthly salaries strictly less than \$20,000.
2. "Average Salary": All monthly salaries in the inclusive range [\$20,000, \$50,000].
3. "High Salary": All monthly salaries strictly greater than \$50,000. The final result table must report all three categories, even if a category contains zero accounts.

Solution:

```
CREATE TABLE Accounts (  
    account_id INT PRIMARY KEY,  
    income INT NOT NULL  
);
```

```
INSERT INTO Accounts (account_id, income) VALUES  
(3, 108939),  
(2, 12747),  
(8, 87709),  
(6, 91738),  
(7, 45169);
```

```
SELECT 'LOW CATEGORY' AS CATEGORY, COUNT(ACCOUNT_ID) AS ACCOUNT_COUNT  
FROM ACCOUNTS  
WHERE INCOME<20000
```

UNION ALL

```
SELECT 'AVERAGE CATEGORY' AS CATEGORY, COUNT(ACCOUNT_ID) AS  
ACCOUNT_COUNT  
FROM ACCOUNTS  
WHERE INCOME>20000 AND INCOME<=50000
```

UNION ALL

```
SELECT 'HIGH CATEGORY' AS CATEGORY, COUNT(ACCOUNT_ID) AS ACCOUNT_COUNT
```

```
FROM ACCOUNTS
WHERE INCOME>50000
```

(B)

Design an Employee Management System database architecture by implementing the following operations sequentially:

1. Create and populate structural tracking tables for Departments, Employees, and Salaries.
2. Query the system using multi-table Joins to pull clear department breakdowns of employees, department and salaries, apply a 10% salary enhancement across all members of the HR department, and delete any individual salary record dropping strictly below \$30,000.
3. List all remaining active employees whose personal salary exceeds the overall company-wide average salary.

Solution:

```
CREATE TABLE Departments(
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50) NOT NULL
);
```

```
CREATE TABLE Employees(
    emp_id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    dept_id INT REFERENCES departments(dept_id)
);
```

```
CREATE TABLE Salaries (
    emp_id INT PRIMARY KEY REFERENCES Employees(emp_id) ON DELETE
    CASCADE,

    salary INT NOT NULL
);
```

```
INSERT INTO Departments (dept_id, dept_name) VALUES
(10, 'HR'), (20, 'IT'), (30, 'Sales');
INSERT INTO Employees (emp_id, name, dept_id) VALUES
(1, 'Alice', 10), (2, 'Bob', 10), (3, 'Charlie', 20), (4, 'David', 20),
(5, 'Emma', 30);
INSERT INTO Salaries (emp_id, salary) VALUES
(1, 50000), (2, 25000), (3, 80000), (4, 40000), (5, 45000);
```

```
SELECT E.*,D.*,S.*
FROM EMPLOYEES E
```

```

JOIN DEPARTMENTS D
ON E.DEPT_ID=D.DEPT_ID
JOIN SALARIES AS S
ON S.EMP_ID=E.EMP_ID;

```

```

UPDATE SALARIES
SET SALARY = SALARY + SALARY * 0.10
WHERE EMP_ID IN(
    SELECT E.EMP_ID
    FROM EMPLOYEES AS E
    JOIN DEPARTMENTS AS D
    ON E.DEPT_ID = D.DEPT_ID
    WHERE D.DEPT_NAME = 'HR'
)
SELECT * FROM EMPLOYEES;

```

```

SELECT EMP_ID
FROM SALARIES
WHERE SALARY>(SELECT AVG(SALARY) FROM SALARIES);

```

(C)

A pizza restaurant stores all the available pizza toppings and their ingredient costs in a table called pizza_toppings.

Every customer must choose exactly 3 different toppings to prepare a pizza.

Write a PostgreSQL query to generate every possible 3-topping pizza combination along with its total ingredient cost.

Requirements

- Every pizza must contain exactly 3 different toppings.
- The same topping cannot appear more than once in a pizza.
- Different arrangements of the same toppings should be considered the same pizza.

Example:

Chicken, Pepperoni, Sausage

Pepperoni, Chicken, Sausage

Sausage, Chicken, Pepperoni

These are all the same pizza, so only one should appear.

- Display the topping names separated by commas.
- Display the total ingredient cost of each pizza.
- Sort the output:

First by highest total cost

If two pizzas have the same cost, sort them alphabetically.

Solution:

```

CREATE TABLE pizza_toppings (

```

```
    topping_name VARCHAR(50) PRIMARY KEY,  
    ingredient_cost NUMERIC(4,2) NOT NULL  
);
```

```
INSERT INTO pizza_toppings (topping_name, ingredient_cost) VALUES  
('Pepperoni', 0.50),  
('Sausage', 0.70),  
('Chicken', 0.55),  
('Onions', 0.25),  
('Extra Cheese', 0.40);
```

```
SELECT  CONCAT(T1.TOPPING_NAME, ',', T2.TOPPING_NAME, ',',  
T3.TOPPING_NAME) AS PIZZA,  
(T1.INGREDIENT_COST+T2.INGREDIENT_COST+T3.INGREDIENT_COST)AS TOTAL_COST  
FROM PIZZA_TOPPINGS AS T1
```

```
JOIN PIZZA_TOPPINGS AS T2  
ON T1.TOPPING_NAME < T2.TOPPING_NAME  
JOIN PIZZA_TOPPINGS AS T3  
ON T2.TOPPING_NAME < T3.TOPPING_NAME  
ORDER BY TOTAL_COST DESC;
```

(D)

Amazon wants to identify returning customers who made another purchase shortly after their first purchase.

Write a PostgreSQL query to find all users who made another purchase within 1 to 7 days of an earlier purchase.

Requirements

1. Compare purchases made by the same user only.
2. Ignore comparisons where the same transaction is matched with itself.
3. The second purchase must occur after the first purchase.
4. Same-day purchases must not be considered.
5. The second purchase must occur within 7 days of the first purchase.
6. Display only the unique User IDs that satisfy the above conditions.

Sort the output in ascending order of user_id.

Solution:

```
CREATE TABLE amazon_transactions (  
    id INT PRIMARY KEY,  
    user_id INT NOT NULL,  
    item VARCHAR(50),  
    created_at TIMESTAMP NOT NULL,  
    revenue INT  
);
```

```
INSERT INTO amazon_transactions (id, user_id, item, created_at, revenue)
VALUES
(1, 101, 'biscuit', '2026-03-01 10:00:00', 12),
(2, 101, 'milk', '2026-03-01 14:00:00', 5),
(3, 101, 'bread', '2026-03-05 09:00:00', 8),
(4, 102, 'banana', '2026-03-10 12:00:00', 4),
(5, 102, 'apple', '2026-03-25 11:00:00', 6),
(6, 103, 'milk', '2026-03-15 08:00:00', 5),
(7, 103, 'bread', '2026-03-16 08:00:00', 7);
```

```
SELECT DISTINCT T1.USER_ID
FROM AMAZON_TRANSACTIONS AS T1
JOIN AMAZON_TRANSACTIONS AS T2
ON T1.USER_ID=T2.USER_ID
WHERE T2.CREATED_AT::DATE>T1.CREATED_AT::DATE
AND T2.CREATED_AT::DATE-T1.CREATED_AT::DATE<=7
ORDER BY T1.USER_ID ASC;
```